

VENDOR PORTAL · AP INVOICE PROCESSING

Two ways to run the Vendor Portal

Serverless · AWS Lambda

vs

Containers · Amazon EKS

Cost, efficiency, growth, releases, backups and the trade-offs of each, explained in plain terms.

WHAT WE HAVE TODAY

Our proof of concept has four parts



Screen

VUE APP

What vendors see:
submit invoices and
track them



Engine

FLASK API

Checks rules, saves
invoices and
documents



Assistant

LANGGRAPH AGENT

Answers vendor
questions in plain
language



Records

POSTGRESQL

Vendors, invoices,
history and chat logs

Today: the Screen and Engine run as containers on Amazon EKS, and the Assistant is heading to Bedrock AgentCore. The scan-to-fill (OCR) feature already runs serverless.

THE TWO OPTIONS IN PLAIN WORDS

A taxi or a leased van



SERVERLESS · LAMBDA

Like taking a taxi

You pay per trip. Someone else owns and services the car. Now and then you wait a moment for pickup.



CONTAINERS · EKS

Like leasing a van

It is always parked outside, ready to go. You pay every month whether you drive or not, and your team does the servicing.

Same destination either way. The difference is **how we pay** and **who looks after the machines**.

SIDE BY SIDE

What changes, part by part

Part	Serverless · Lambda	Containers · EKS
Screen (Vue)	Built once into files and served from S3 storage	Small web server in a container on EKS (as today)
Engine (Flask API)	Reshaped into Lambda functions behind API Gateway	Runs as-is in a container on EKS (as today)
Assistant (agent)	Bedrock AgentCore Runtime — the same in both	Bedrock AgentCore Runtime — the same in both
Records (Postgres)	Amazon RDS plus RDS Proxy to handle many short connections	Amazon RDS (as today)
Invoice uploads	Vendor uploads straight to S3 with a one-time link	Sent through the Engine (as today)

The Assistant, the database and the stored documents stay the same in both. Only the Screen, the Engine and uploads change.

COST

What we pay for

SERVERLESS · LAMBDA

Pay only when it is used

- Close to zero cost when nobody is using the portal
- Billed per request and per second of work
- Free allowance of 1 million Lambda requests a month
- Extras: API Gateway per call, RDS Proxy, private access setup

CONTAINERS · EKS

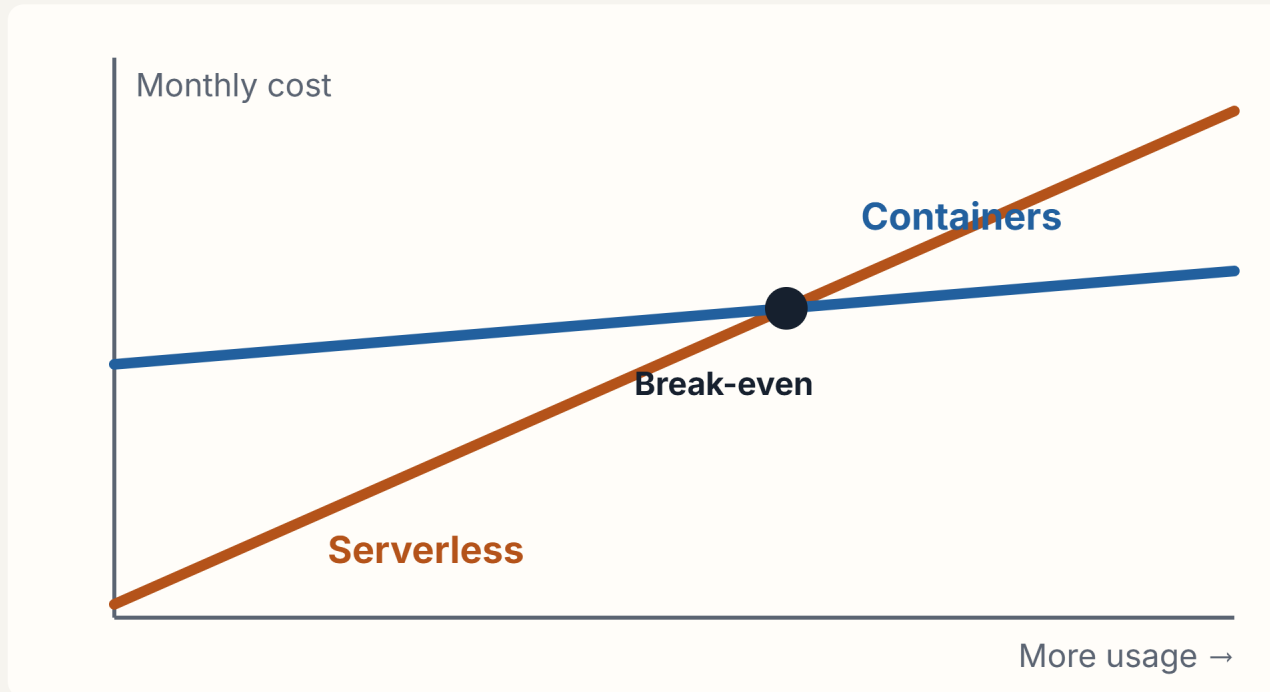
Pay for capacity, used or not

- EKS cluster fee of about US\$73 a month, before any servers
- Servers run 24/7, even at night and on weekends
- Cheaper per request once traffic is steady and high
- Much cheaper if we join an existing shared cluster

Same in both: database, AI assistant, document storage and AI model usage. Our estimate: Serverless **[US\$__]** / month vs Containers **[US\$__]** / month.

COST OVER USAGE

Where the cost lines cross



Left of the crossing, serverless is cheaper

A portal used mostly in office hours by a known group of vendors usually sits here.

Right of it, containers win

Heavy, round-the-clock use, or a cluster we already share, moves us here.

Illustrative only, not to scale. Our break-even point depends on vendor numbers and usage hours: [to be estimated].

EFFICIENCY

Effort to build and to run

SERVERLESS · LAMBDA

Less upkeep later, more rework now

- AWS patches and runs the servers for us
- The Flask Engine must be reshaped into functions
- Uploads, database links and long tasks need redesign
- Harder to run the whole portal on a laptop

CONTAINERS · EKS

Works as built, more upkeep

- Already running on EKS: nothing to rework to launch
- The same package runs on a laptop, in testing and live
- Team patches servers and upgrades Kubernetes regularly
- Needs Kubernetes skills on the team

Bottom line: serverless saves effort over time; containers save effort right now. Rework estimate for serverless: [__] weeks.

SCALABILITY

Handling busy days

SERVERLESS · LAMBDA

Grows instantly, per request

- Each request gets its own copy, up and down automatically
- First request after a quiet spell can be slower (a cold start)
- Default limit of 1,000 copies at once, can be raised
- Needs RDS Proxy so the database is not overwhelmed

CONTAINERS · EKS

Grows in steps, by rules we set

- Adds copies and servers when busy, using rules we tune
- Always warm: no slow first request
- A sudden spike can take a few minutes to add servers
- Steady response times for long AI conversations

Bottom line: vendor traffic is modest and predictable, so both cope well. Serverless needs no tuning; containers feel smoother.

CI/CD

Releasing changes

Same Jenkins pipeline shape in both. Only the last two steps differ.



SERVERLESS · LAMBDA

Many small pieces, instant rollback

- Each function is packaged and versioned on its own
- Shift traffic to a new version gradually
- Roll back in seconds by pointing to the old version

CONTAINERS · EKS

One package, same everywhere

- One container image stored in ECR
- Rolling update swaps copies one at a time
- Roll back by redeploying the previous image

Keeping data safe, getting back up

Same in both

DATABASE

RDS daily backups and point-in-time restore

INVOICE DOCUMENTS

S3 storage with versioning

CODE

Git plus stored release packages

SERVERLESS · LAMBDA

Little to rebuild

- No servers to back up: the app is code plus settings
- Rebuild in another region by redeploying templates
- Old versions are kept for one-step rollback

CONTAINERS · EKS

More pieces to restore

- The cluster, its add-ons and settings must be rebuilt too
- Setup scripts (Terraform, Helm) must stay up to date
- Recovery drills take longer and need more skill

Keeping the portal private

SERVERLESS · LAMBDA

AWS carries more of the load

- AWS secures and patches the underlying servers
- A private API Gateway keeps the Engine off the internet
- Hosting the Screen privately needs extra setup
- Each function gets its own narrow permissions

CONTAINERS · EKS

Proven today, more to manage

- Private load balancer on EKS is already working
- We patch server images and Kubernetes ourselves
- Image scanning in Jenkins catches known flaws
- More settings to get right: network rules and roles

Must-do in both: keep VPN-only access, and add real vendor sign-in before go-live.

Pros and cons

Pros

- Lowest cost when traffic is low or uneven
- No servers to patch or upgrade
- Scales automatically, no tuning
- Fast recovery and instant rollback
- Test environments cost almost nothing when idle

Cons

- Rework of the current Engine and uploads
- Slower first response after quiet periods
- Hard limits: 6 MB per request, 15 minutes per task
- Tied closely to AWS
- Harder to test everything on a laptop

Pros and cons

Pros

- Runs today with no rework
- Same package from laptop to production
- Always warm, steady response times
- Portable to any cloud or on-premise Kubernetes
- No request size or time limits

Cons

- Pays for servers even when idle
- Cluster fee plus regular upgrades
- Needs Kubernetes skills on the team
- More pieces to back up and restore
- Scaling rules need tuning

SCORECARD

How they compare at a glance

Area	Serverless · Lambda	Containers · EKS
Cost when usage is low	Lower	Higher
Cost when usage is high and steady	Higher	Lower
Effort to launch from today	More rework	Ready now
Day-to-day upkeep	Low	Higher
Sudden spikes	Instant	Takes minutes
Response speed	Occasional slow start	Consistently fast
Releases and rollback	Seconds	Minutes
Backup and recovery	Simpler	More steps
Portability	AWS only	Anywhere

Bold marks the stronger option in each row.

THE REAL QUESTION

The assistant, the database and the data stay the same. We are choosing how to pay for, and look after, the screen and the engine.

HOW TO DECIDE

Which fits us better

SERVERLESS · LAMBDA

Choose serverless if...

- Use is mostly office hours, with quiet nights and weekends
- There is no shared EKS cluster for us to join
- We want the least infrastructure to look after
- We can fund the rework before go-live

CONTAINERS · EKS

Choose containers if...

- A shared company EKS cluster is available to us
- We want to go live soon with what is built
- Traffic will be steady or large
- We may move clouds or run on-premise later

Middle path: keep the containers but run them on AWS Fargate (ECS), so there are no servers or Kubernetes to manage and no rework.

NEXT STEPS

What we need to decide

- 1 Confirm usage** Number of vendors, peak days and usage hours
- 2 Check the platform** Is there a shared company EKS cluster we can join?
- 3 Price all three** AWS Pricing Calculator for Lambda, EKS and Fargate
- 4 Decide** Leadership choice by [date]
- 5 Either way** Add real vendor sign-in before go-live